

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..49

CONFIGURATION

 Copy as Markdown

Configuration

The `withServerpod` helper takes optional named arguments that control how the test server starts and how tests are isolated from each other.

Some of the values below, such as `ServerpodRunMode` and `Constant`, come from `package:serverpod/serverpod.dart` rather than the generated test tools, so import it when you use them. The `serverDirectory` option takes a `Directory` from `dart:io`.

```
withServerpod(  
  'Given Products endpoint',  
  (sessionBuilder, endpoints) {  
    /* test code */  
  },  
  rollbackDatabase: RollbackDatabase.afterAll,  
);
```

Options

Option	What it does	Default
<code>applyMigrations</code>	Whether pending migrations are applied when the test server starts.	<code>true</code>
<code>configOverride</code>	A function that changes values in the loaded config before the server starts.	<code>null</code>
<code>databaseInterceptor</code>	A database interceptor for the test server to use.	<code>null</code>



Option	What it does	Default
<code>enableSessionLogging</code>	Whether session logging is on.	<code>false</code>
<code>experimentalFeatures</code>	Experimental features to enable, such as diagnostic event handlers .	<code>null</code>
<code>rollbackDatabase</code>	When database changes are rolled back. See below .	<code>RollbackDatabase.afterEach</code>
<code>runMode</code>	The run mode the server starts in.	<code>test</code>
<code>runtimeParametersBuilder</code>	Runtime parameters to apply before startup. See runtime parameters .	<code>null</code>
<code>serverDirectory</code>	The directory that <code>config/<runMode>.yaml</code> , <code>config/passwords.yaml</code> , and <code>migrations/</code> are resolved against.	The directory the test process runs in
<code>serverpodLoggingMode</code>	The logging mode used when creating Serverpod.	<code>ServerpodLoggingMode.normal</code>
<code>serverpodStartTimeout</code>	How long to wait for the server to start.	120 seconds
<code>testGroupTagsOverride</code>	The tags applied to the generated test group. See below .	<code>['integration']</code>
<code>testServerOutputMode</code>	How much server output reaches your terminal. See below .	<code>TestServerOutputMode.normal</code>

NOTE

When `config/generator.yaml` sets `features: database: false`, the generated `withServerpod` has no `applyMigrations`, `databaseInterceptor`, `rollbackDatabase`, or `runtimeParametersBuilder` parameter, so passing one is a compile error. Rollback is off in that case and there is nothing to migrate.

Projects created without a database, for example with `--no-database`, don't include this setting, so add it and run `serverpod generate`.

Set `serverDirectory` when tests run from somewhere other than the server package, such as the workspace root. Without it the server resolves those paths against the current directory, misses your config, and falls over later on whatever it needed from it.

Run mode

Tests run in the `test` run mode, so the server reads `config/test.yaml`. Point them at a different mode when you need its configuration instead:

```
withServerpod(  
  'Given Products endpoint',  
  (sessionBuilder, endpoints) {  
    /* test code */  
  },  
  runMode: ServerpodRunMode.development,  
);
```

Database rollback

By default each test runs inside a transaction that is rolled back when that test ends, so tests cannot see each other's data. Three values control this:

Value	Rolls back
<code>RollbackDatabase.afterEach</code>	After every test. The default.
<code>RollbackDatabase.afterAll</code>	Once the whole <code>withServerpod</code> group finishes.
<code>RollbackDatabase.disabled</code>	Never. Data stays until the group finishes and its database is dropped.

Three situations call for changing it.

Tests that build on each other. When one test depends on what the previous one left behind, `afterAll` keeps state across the group and still rolls everything back at the end. Tests that share state are usually worth avoiding, but this helps when the setup is expensive.

Concurrent database work in an endpoint. The rollback wraps each test in one transaction, and concurrent calls cannot be nested inside it. An endpoint that runs database calls or transactions in parallel throws

`InvalidConfigurationException` unless you pass `RollbackDatabase.disabled`:

```
Future<void> concurrentTransactionCalls(Session session) async {
  await Future.wait([
    session.db.transaction((tx) => /*...*/),
    // Throws InvalidConfigurationException unless rollbackDatabase
    // is RollbackDatabase.disabled.
    session.db.transaction((tx) => /*...*/),
  ]);
}
```

With rollback disabled, later tests in the same group see what earlier tests committed. To start each test from a clean state, clean up in `tearDown`:

```
withServerpod(
  'Given ProductsEndpoint when calling concurrentTransactionCalls',
  (sessionBuilder, endpoints) {
    tearDown(() async {
      var session = sessionBuilder.build();
      await Product.db.deleteWhere(session, where: (_) => Constant.bool(true));
    });

    test('then it commits all transactions', () async {
      var result =
        await endpoints.products.concurrentTransactionCalls(sessionBuilder);
      // ...
    });
  },
  rollbackDatabase: RollbackDatabase.disabled,
);
```

Caught database exceptions. This is the one case where the test tools behave differently from production:

```
var transactionFuture = session.db.transaction((tx) async {
  var data = UniqueData(number: 1, email: 'test@test.com');
  try {
    await UniqueData.db.insertRow(session, data, transaction: tx);
    await UniqueData.db.insertRow(session, data, transaction: tx);
  } on DatabaseException catch (_) {
    // Ignored.
  }
});

// Throws in production, but not under the test tools.
await transactionFuture;
```

In production a transaction fails if any database exception happened inside it, even one your code caught. The test tools emulate nested transactions and do not, so code that swallows database exceptions passes a test it would fail in production. Set `RollbackDatabase.disabled` to get production behavior.

Server output

The `testServerOutputMode` option controls what the test server prints, which is the setting to reach for when you cannot see why a server failed:

Value	Shows
<code>TestServerOutputMode.normal</code>	Errors only. The default.
<code>TestServerOutputMode.verbose</code>	Everything the server prints.
<code>TestServerOutputMode.silent</code>	Nothing.

Test tags

Test groups created by `withServerpod` are tagged `integration`, which is what lets you run integration and unit tests separately. Change the tag with `testGroupTagsOverride`. See [running tests separately](#).

Related

- [Writing tests](#): the pieces every test uses.
- [Advanced examples](#): the options above put to work.
- [Server configuration](#): the config files the test server reads.